

Entrega Final: Compilador Claudio a Swift

Gramatica ampliada, reglas semanticas, SDT de traduccion, implementacion y pruebas

Curso: Teoria de Compiladores

Autores: Juan David Cardenas Jimenez, Maria Jose Restrepo Ramirez, Jose Zuluaga

Junio de 2026

Convencion visual exigida por el enunciado

Rojo: producciones originales de la gramatica.

Amarillo: reglas semanticas del Quiz 4 con atributos, dominios y acciones.

Azul: acciones SDT que sintetizan codigo Swift.

1. Alcance Del Compilador Final

La entrega integra las cuatro fases del compilador: analisis lexico, analisis sintactico, analisis semantico y traduccion dirigida por sintaxis hacia Swift. La salida destino solo se produce cuando las fases determinísticas aceptan el programa fuente.

Requisito del enunciado	Evidencia en Claudio
Lexico -> sintactico -> semantico -> SDT	Endpoint /api/compilar encadena las fases y bloquea Swift si alguna falla.
Errores unificados	Cada diagnostico final trae fase, fila, columna, lexema y mensaje.
Tabla de simbolos disponible durante SDT	AnalizadorSemantico construye tabla de simbolos y /api/compilar la retorna junto con la salida.
Tres archivos de prueba	tests/final/caso_valido.claudio, caso_semantico.claudio, caso_lexico_sintactico.claudio.
Bonus IA	OpenAI valida la salida Swift generada; si no hay API key, el compilador sigue funcionando.

Cadena de fases:

```
Fuente Claudio
-> Lexer: tokens + errores lexicos
-> Parser RD: arbol + errores sintacticos
-> Analizador semantico: tabla de simbolos + SEM-1..SEM-7
-> SDT Swift: codigo destino + mapeo linea a linea
-> IA opcional: revision del Swift generado
```

2. Gramatica Base y SDT

Las producciones estan escritas en forma BNF compacta. Los no terminales con repeticion usan recursividad derecha o epsilon; sus acciones SDT concatenan fragmentos Swift en orden de aparicion.

2.1 Programa y declaraciones

```
programa -> declaracion programa | ε
declaracion -> importacion | decl_variable | def_funcion | def_clase | sentencia
importacion -> importar ID
```

SDT-PROG

programa.swift = declaracion.swift + "\n" + programa1.swift.

programa -> ε sintetiza cadena vacia; no altera la salida.

importacion.swift = "import " + ID.lexema.

2.2 Variables y tipos

```
decl_variable -> (var | sea) tipo ID = expresion
tipo -> entero | real | cadena | booleano | ID
tipo_basico -> entero | real | cadena | booleano
```

SEM-1 y SEM-4: declaracion de variable

Atributos: ID.nombre, tipo.declarado, expresion.tipo, ambito.actual, tabla_simbolos

Dominio / restriccion: ID no debe existir en el ambito actual; expresion.tipo debe ser compatible con tipo.declarado. entero puede promoverse a real.

Accion: Reportar duplicado o incompatibilidad; si no hay duplicado, registrar simbolo con tipo e inmutabilidad.

SDT-VAR

Si el declarador es var: decl_variable.swift = "var ID: TipoSwift = expr.swift".

Si el declarador es sea: decl_variable.swift = "let ID: TipoSwift = expr.swift".

TipoSwift(entero, real, cadena, booleano) = Int, Double, String, Bool.

2.3 Funciones y parametros

```
def_funcion -> funcion tipo_ret ID ( parametros ) hacer bloque fin_funcion
tipo_ret -> tipo_basico | ε
parametros -> param_lista | ε
param_lista -> tipo ID param_resto
param_resto -> , tipo ID param_resto | ε
sent_retornar -> retornar expresion
```

SEM-FUNC: ambito y parametros

Atributos: funcion.nombre, tipo_ret, parametros.lista, ambito.funcion

Dominio / restriccion: El nombre de funcion no debe duplicarse en el ambito actual; cada parametro se declara en el ambito de la funcion.

Accion: Registrar la funcion como simbolo, abrir ambito, registrar parametros, analizar bloque y cerrar ambito.

SDT-FUNC

def_funcion.swift = "func ID(params.swift) -> TipoRet {\n" + bloque.swift + "\n}".

Si tipo_ret es ε, se omite "-> TipoRet".

param_lista.swift = "_ ID: TipoSwift" + param_resto.swift; param_resto epsilon sintetiza cadena vacia.

sent_retornar.swift = "return " + expresion.swift.

2.4 Clases, atributos y metodos

```
def_clase -> clase ID herencia_opt hacer cuerpo_clase fin_clase
herencia_opt -> hereda ID | ε
cuerpo_clase -> miembro_clase cuerpo_clase | ε
miembro_clase -> def_atributo | def_metodo
def_atributo -> atributo tipo ID
def_metodo -> metodo ID ( parametros ) hacer bloque fin_funcion
```

SEM-CLASE: nombres de clase, atributos y metodos

Atributos: clase.nombre, atributo.nombre, metodo.nombre, ambito.clase

Dominio / restriccion: Cada nombre declarado dentro de la clase debe ser unico en el ambito de clase.

Accion: Registrar clase, abrir ambito de clase, registrar atributos/metodos y analizar cuerpos de metodo.

SDT-CLASS

def_clase.swift = "class ID[: Super] {\n" + cuerpo.swift + "\n}".

def_atributo.swift = "var ID: TipoSwift = default(TipoSwift)" para evitar propiedades almacenadas sin valor inicial.

default(Int, Double, String, Bool) = 0, 0.0, "", false.

def_metodo.swift = "func ID(params.swift) {\n" + bloque.swift + "\n}".

2.5 Bloques y sentencias

```
bloque -> sentencia bloque' bloque' -> sentencia bloque' | ε
sentencia -> sent_id | sent_si | sent_para | sent_mientras
| sent_retornar | sent_imprimir | sent_romper | sent_continuar | decl_variable
sent_id -> (ID | este) resto_id
resto_id -> = expresion | . ID resto_id | ( argumentos ) | ε
argumentos -> arg_lista | ε
arg_lista -> expresion argresto
argresto -> , expresion argresto | ε
```

SEM-2, SEM-3 y SEM-5: uso y asignacion

Atributos: ID.nombre, ID.tipo, ID.inmutable, expresion.tipo, tabla_simbolos

Dominio / restriccion: El identificador debe existir; una constante declarada con sea no puede reasignarse; el tipo asignado debe ser compatible.

Accion: Reportar identificador no declarado, reasignacion de constante o incompatibilidad de tipo.

SDT-STMT

sent_id asignacion: "lhs.swift = expresion.swift"; este.campo se traduce a self.campo.

sent_id llamada: "ID(args.swift)"; acceso por punto conserva el receptor y traduce este a self.

argumentos y restos con epsilon sintetizan cadena vacia; las repeticiones agregan coma y expresion.swift.

sent_imprimir.swift = "print(" + expresion.swift + ")"; romper/continuar -> break/continue.

2.6 Condicionales y ciclos

```
sent_si -> si expresion entonces bloque rama_sino fin_si
rama_sino -> sino bloque | ε
sent_para -> para ID desde expresion hasta expresion paso_opt hacer bloque fin_para
paso_opt -> paso expresion | ε
sent_mientras -> mientras expresion hacer bloque fin_mientras
```

SEM-6 y SEM-7: condiciones y limites

Atributos: condicion.tipo, desde.tipo, hasta.tipo, paso.tipo

Dominio / restriccion: La condicion de si/mientras debe ser booleana; desde, hasta y paso de para deben ser enteros o reales.

Accion: Reportar condicion no booleana o limites/paso no numericos. La variable de para se declara como entero en su ambito.

SDT-CONTROL

sent_si.swift = "if cond.swift {\n" + bloque.swift + "\n}" + rama_sino.swift.

rama_sino epsilon sintetiza cadena vacia; rama_sino con sino sintetiza " else {\n bloque.swift \n}".

sent_para.swift = "for ID in stride(from: desde.swift, through: hasta.swift, by: paso.swift) {\n bloque.swift \n}".

paso_opt epsilon sintetiza 1; sent_mientras.swift = "while cond.swift {\n bloque.swift \n}".

2.7 Expresiones

```

expresion -> expr_or
expr_or -> expr_and expr_or' expr_or' -> o expr_and expr_or' | ε
expr_and -> expr_rel expr_and' expr_and' -> y expr_rel expr_and' | ε
expr_rel -> expr_add expr_rel' expr_rel' -> op_rel expr_add | ε
expr_add -> expr_mul expr_add' expr_add' -> + expr_mul expr_add' | - expr_mul expr_add' | ε
expr_mul -> expr_pot expr_mul' expr_mul' -> * expr_pot expr_mul' | / expr_pot expr_mul' | % expr_pot
expr_mul' | ε
expr_pot -> expr_unaria expr_pot' expr_pot' -> ** expr_unaria expr_pot' | ε
expr_unaria -> no expr_unaria | - expr_unaria | expr_primaria
expr_primaria -> literal | verdadero | falso | nulo | nuevo ID(argumentos) | este sufijo_id | ID sufijo_id |
( expresion )
sufijo_id -> ( argumentos ) | . ID sufijo_id | ε

```

SDT-EXPR

Operadores aritmeticos y relacionales conservan su simbolo Swift: +, -, *, /, %, ==, !=, <, >, <=, >=.

y -> &&, o -> ||, no -> !. verdadero/falso/nulo -> true/false/nil.

nuevo ID(args) -> ID(args). este.sufijo -> self.sufijo.

a ** b -> pow(Double(a), Double(b)); los nodos epsilon de expresion sintetizan cadena vacia.

3. Reglas Semanticas Implementadas

Regla	Validacion	Modulo
SEM-1	Declaracion duplicada en el mismo ambito.	backend/semantico.py
SEM-2	Uso de identificador no declarado.	backend/semantico.py
SEM-3	Reasignacion de constante declarada con sea.	backend/semantico.py
SEM-4	Tipo incompatible en declaracion.	backend/semantico.py
SEM-5	Tipo incompatible en asignacion.	backend/semantico.py
SEM-6	Condicion de si/mientras no booleana.	backend/semantico.py
SEM-7	Limites desde/hasta/paso del para no numericos.	backend/semantico.py

La tabla de simbolos almacena nombre, tipo, inmutabilidad, estado de inicializacion, ambito, fila y columna. La inferencia de tipos es conservadora: si una expresion no se puede determinar con certeza, se evita un falso positivo semantico.

4. Implementacion En El Repositorio

Modulo	Responsabilidad
backend/lexer.py	Analisis lexico, tokens, errores lexicos y posiciones.
backend/parser_rd.py	Parser descendente recursivo, AST y recuperacion sintactica.
backend/parser_ll1.py	Parser predictivo LL(1), FIRST/FOLLOW, tabla y traza.
backend/semantico.py	Reglas SEM-1..SEM-7 y tabla de simbolos.
backend/traductor.py	Acciones SDT practicas Claudio -> Swift y mapeo linea a linea.
backend/ai_suggestions.py	OpenAI para sugerencias y validacion del Swift generado con fallback.
backend/main.py	API FastAPI; /api/compilar encadena la entrega final.
frontend/src/components/swift-view.tsx	Visualizacion mejorada del lenguaje destino Swift.

Contrato de /api/compilar:

```
POST /api/compilar { "codigo": "..."}
Respuesta:
  valido: boolean
  swift: string
  mapeo: [{ claudio, swift }] // vacio si hay errores
  errores: [{ fase, fila, columna, lexema, mensaje, regla?, esperado?, sugerencia? }]
  tabla_simbolos: [...]
  lexico, sintactico, semantico
  validacion_ia
```

5. Pruebas Obligatorias

Archivo	Objetivo	Resultado esperado
tests/final/caso_valido.claudio	Programa correcto con funcion, constante, ciclo para, acumulador, imprimir y condicional.	valido=true; swift no vacio; mapeo no vacio; tabla de simbolos con datos.
tests/final/caso_semantico.claudio	Sintaxis correcta con SEM-3, SEM-4 y SEM-6.	valido=false; swift=""; fase semantico en errores.
tests/final/caso_lexico_sintactico.claudio	Falta entonces, parentesis sin cerrar y caracter @.	valido=false; swift=""; tabla semantica vacia; fases lexico/sintactico.

Comando de verificacion:

```
cd backend
python3 -m unittest test_quiz3.py test_quiz3_diagnostic_cases.py test_quiz4_semantico.py test_entrega_final.py

cd ../frontend
npm run lint
npm run build
```

6. Bonus IA Con OpenAI

Se implementa una modalidad complementaria: OpenAI revisa el código Swift generado solo después de que el compilador determinístico acepta el programa. La IA no decide si Claudio es válido; solo resume problemas o sugerencias sobre la salida destino.

Modalidad y fallback

Variable usada: OPENAI_API_KEY. Modelo por defecto: OPENAI_MODEL o gpt-5.4-mini.

Si no hay API key o falla la llamada, validacion_ia.estado_ia queda en no_disponible/error y la salida Swift determinística se conserva.

Prompt: revisar si el Swift generado parece sintácticamente razonable, conserva la intención Claudio y mantiene estructuras balanceadas.

```
Ejemplo de respuesta IA sin API key:
validacion_ia = {
  "estado_ia": "no_disponible",
  "valido": null,
  "resumen": "Configura OPENAI_API_KEY en el backend para activar la validacion IA del Swift generado.",
  "problemas": [],
  "sugerencias": []
}
```

7. Conclusiones

- La entrega final queda cerrada como un flujo de compilación completo: las fases no son pantallas aisladas, sino una cadena con compuertas claras.
- La gramática, las reglas semánticas y las acciones SDT se mantienen alineadas con la implementación real: tipos, bloques, ciclos, funciones, clases y expresiones tienen traducción definida.
- La visualización Swift facilita defender el resultado: muestra salida destino, mapeo fuente-destino, estado de validación y bloqueo explícito cuando hay errores.
- OpenAI se conserva como bonus educativo y no como dependencia crítica: el compilador sigue funcionando aunque la IA no esté disponible.